House Number Recognition Using CNN and the SVHN Dataset

Final Machine Learning Project Report

Al-Aqsa University
Faculty of Information Technology
Department of Computer and Information Sciences

Students
Abdelrahman Ahmed Alazzazi
Haitham Hani Dalloul

Supervisor:

Dr. Hassan S. Elhallaq

Abstract

This project presents a simple Convolutional Neural Network (CNN) for classifying cropped house-number images from the Street View House Numbers (SVHN) dataset. The task is a supervised multiclass classification problem with ten output classes representing digits 0 to 9. The original training set was divided into 80% training data and 20% validation data, while the original test set was kept for final evaluation. The model contains two convolution layers, two max-pooling layers, one dense hidden layer, dropout, and a softmax output layer. It was trained using the Adam optimizer for a maximum of 20 epochs with Early Stopping. The final model achieved 87.92% accuracy on 26,032 test images, with a macro F1-score of 0.8679. The results demonstrate that a compact CNN can provide good performance on real-world digit images while remaining suitable for a student-level machine learning project.

Keywords: SVHN, CNN, image classification, supervised learning, digit recognition.

1. Introduction

Recognizing digits from natural images is more difficult than recognizing clean handwritten digits. Street-view images can contain different lighting conditions, complex backgrounds, blur, camera angles, and neighboring digits. The SVHN dataset provides cropped color images of house-number digits collected from Google Street View. This makes it useful for studying image classification under realistic conditions.

The project focuses on classifying the main digit in each cropped 32×32 RGB image. It does not detect digits inside a full street image and it does not read a complete multi-digit address in one step.

2. Problem Statement and Objectives

The problem is to learn a mapping from an RGB image to one of ten digit classes. Each input image has a known class label, so the task is supervised multiclass classification.

The main objectives are:

- Load and understand the SVHN dataset.
- Analyze image samples and the distribution of digit classes.
- Prepare training, validation, and test data.
- Build a small CNN that can be explained and implemented by students.
- Evaluate the model using accuracy, loss, per-class measures, and a confusion matrix.
- Discuss errors, limitations, and possible improvements.

3. Dataset Description

The project uses the cropped version of the Street View House Numbers dataset. Each image is a 32×32 color image with three RGB channels. The dataset contains ten classes corresponding to digits 0 through 9.

Item	Value
Image size	32 × 32 × 3
Number of classes	10
Original training set	73,257 images
Project training set	58,606 images
Validation set	14,651 images
Test set	26,032 images
Pixel data type	uint8
Original pixel range	0 to 255



Figure 1. Sample images from the SVHN training set.

The visual samples show that some digits are clear while others are blurred, partially hidden, or surrounded by neighboring digits. These variations increase the difficulty of the task.

4. Exploratory Data Analysis

The number of training examples differs across classes. Digit 1 is the largest class, while digit 9 is the smallest class in the selected training split.

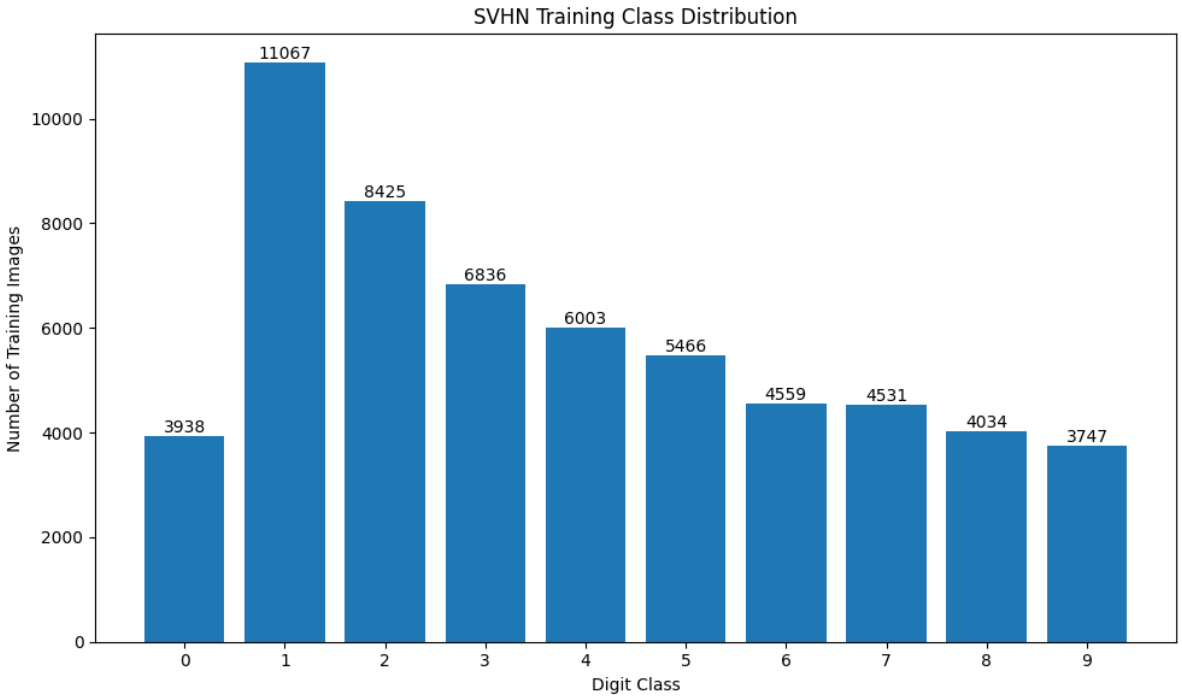


Figure 2. Distribution of digit classes in the training split.

Digit	Images	Percentage
0	3,938	6.72%
1	11,067	18.88%
2	8,425	14.38%
3	6,836	11.66%
4	6,003	10.24%
5	5,466	9.33%
6	4,559	7.78%
7	4,531	7.73%
8	4,034	6.88%
9	3,747	6.39%

The class distribution is not perfectly balanced. This is considered when interpreting per-class Recall and F1-score rather than relying only on total accuracy.

5. Data Preparation

- Data split: 80% of the original training set was used for training and 20% for validation.
- Shuffling: training images were shuffled at each epoch.
- Batching: images were processed in batches of 64.
- Prefetching: the next batch was prepared while the current batch was processed.

- Normalization: a Rescaling layer converted pixel values from 0-255 to 0-1.

The validation and test sets were not shuffled because their order does not affect evaluation. No images from the test set were used to update the model weights.

6. CNN Architecture

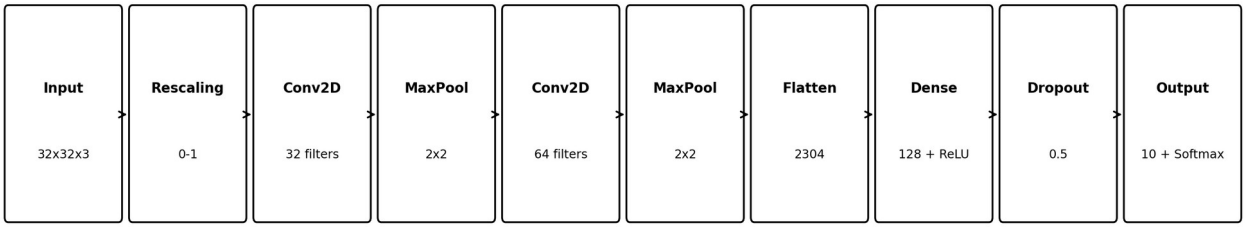


Figure 3. Architecture of the CNN used in the project.

Layer	Output shape	Parameters	Purpose
Rescaling	32 × 32 × 3	0	Normalize pixel values
Conv2D (32)	30 × 30 × 32	896	Extract basic visual features
MaxPooling	15 × 15 × 32	0	Reduce spatial dimensions
Conv2D (64)	13 × 13 × 64	18,496	Extract higher-level features
MaxPooling	6 × 6 × 64	0	Reduce spatial dimensions
Flatten	2,304	0	Convert feature maps to a vector
Dense (128)	128	295,040	Combine learned features
Dropout (0.5)	128	0	Reduce overfitting
Dense (10)	10	1,290	Produce class probabilities

The model contains 315,722 trainable parameters. ReLU is used in the hidden layers and Softmax is used in the output layer because the classes are mutually exclusive.

7. Training Configuration

Setting	Value
Optimizer	Adam
Learning rate	0.001
Loss function	Sparse Categorical Crossentropy
Batch size	64
Maximum epochs	20
Early Stopping patience	3 epochs
Monitored value	Validation loss
Random seed	42

Early Stopping restored the model weights from epoch 18 because that epoch produced the lowest validation loss. Training continued to epoch 20 because the patience condition had not yet been exceeded.

8. Results

8.1 Training curves

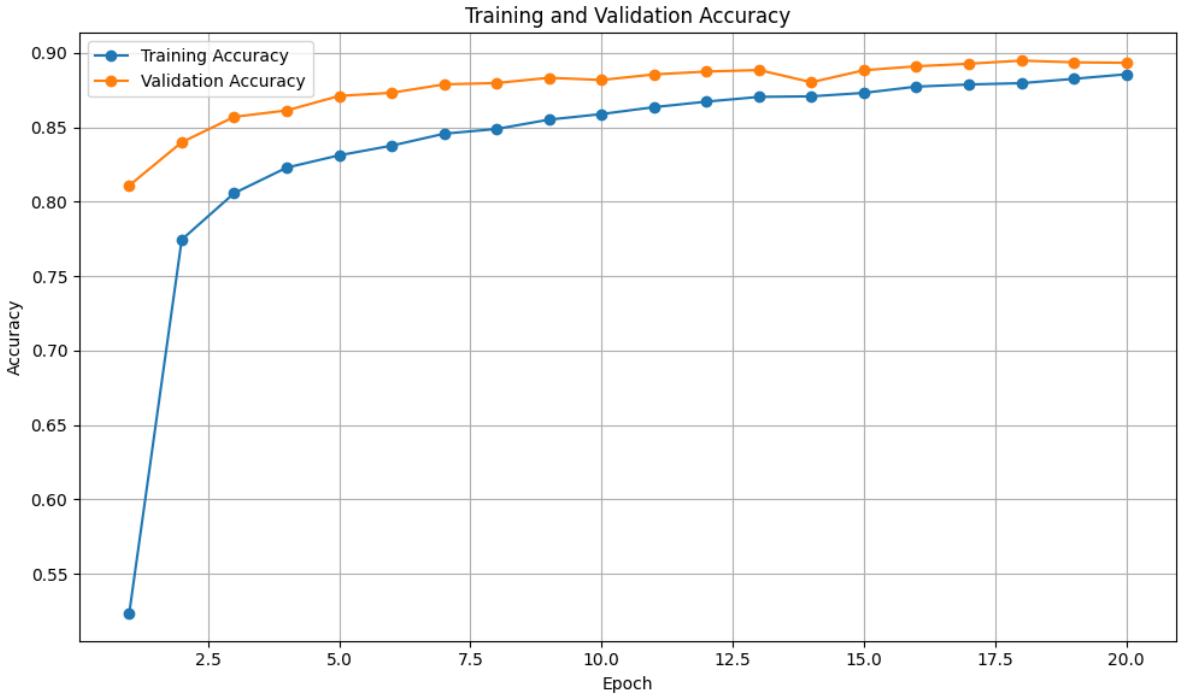


Figure 4. Training and validation accuracy across 20 epochs.

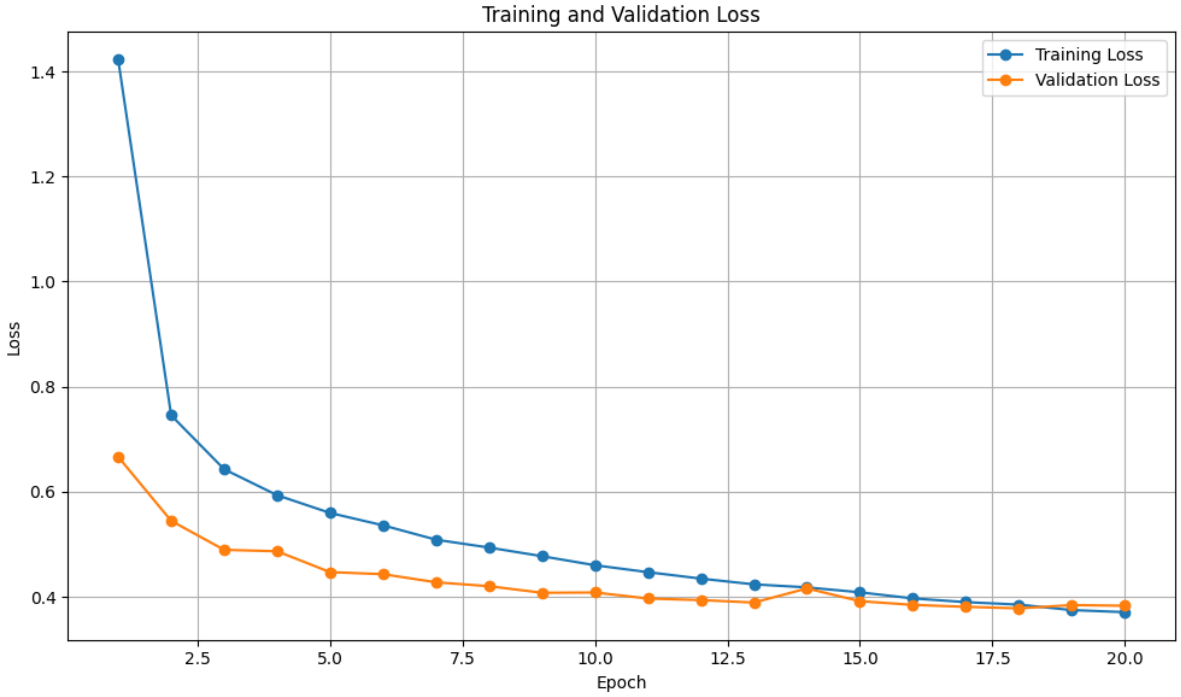


Figure 5. Training and validation loss across 20 epochs.

Accuracy improved strongly during the first epochs and then increased more gradually. The lowest validation loss was 0.3780 at epoch 18. Validation accuracy was slightly higher than training accuracy during much of training because Dropout is active on training batches but disabled during validation.

8.2 Final test results

Measure	Result
Test accuracy	87.92%
Test loss	0.4334
Macro precision	0.8708
Macro recall	0.8665
Macro F1-score	0.8679
Weighted F1-score	0.8788

The final test accuracy is lower than the best validation accuracy, but the difference is small enough to indicate reasonable generalization to unseen data.

8.3 Per-class performance

Digit	Precision	Recall	F1-score	Support
0	0.8963	0.8670	0.8814	1,744
1	0.9320	0.9186	0.9252	5,099
2	0.8936	0.9308	0.9118	4,149
3	0.8535	0.8185	0.8356	2,882
4	0.8721	0.9029	0.8872	2,523
5	0.8343	0.8914	0.8619	2,384
6	0.8643	0.8250	0.8442	1,977
7	0.8837	0.8995	0.8915	2,019

Digit	Precision	Recall	F1-score	Support
8	0.8814	0.7578	0.8119	1,660
9	0.8045	0.8539	0.8285	1,595

Digit 1 achieved the highest F1-score (0.9252). Digit 8 had the lowest Recall (0.7578) and the lowest F1-score (0.8119), meaning that a larger proportion of true digit-8 images were confused with other classes.

8.4 Confusion matrix

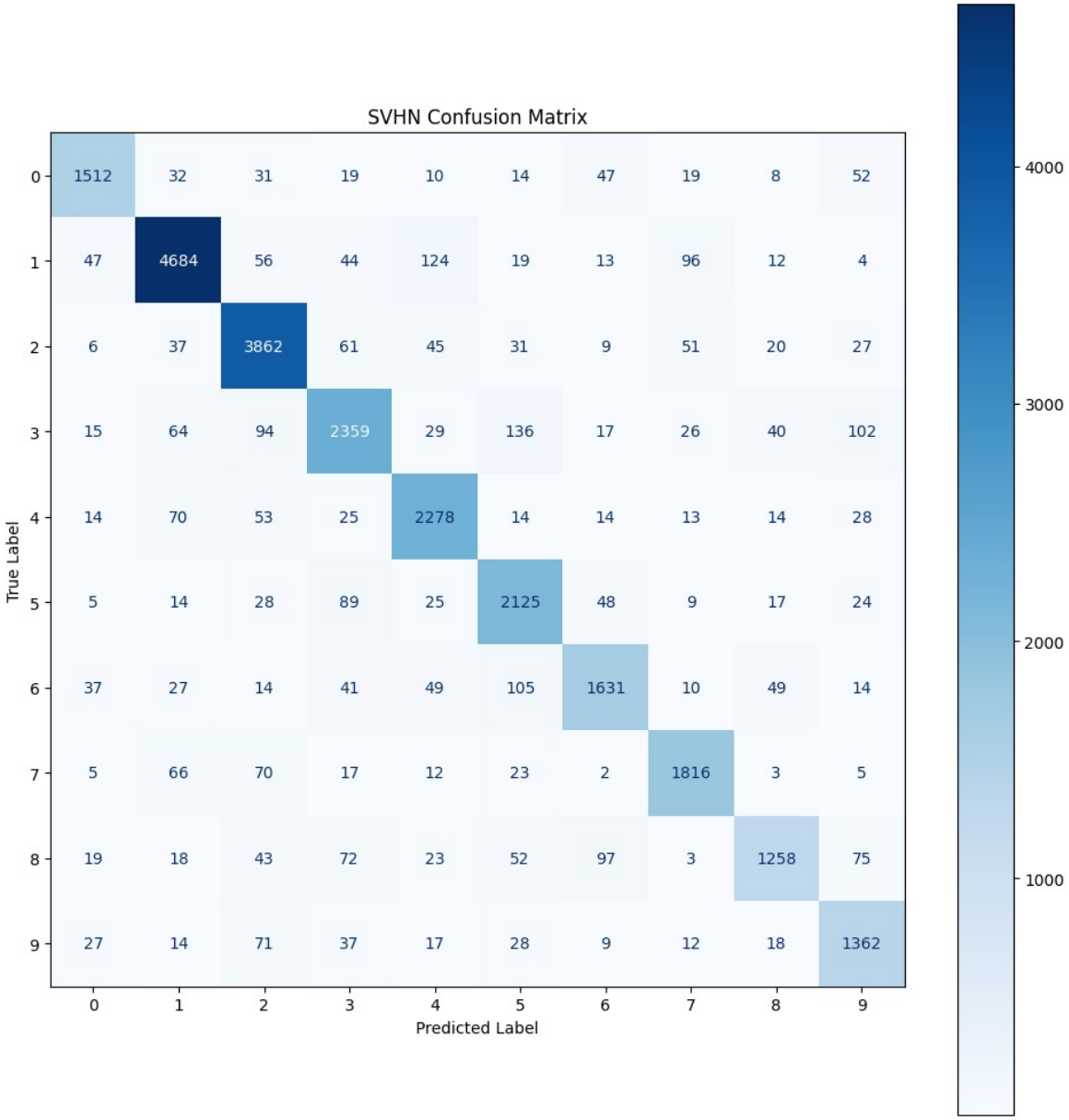


Figure 6. Confusion matrix on the test set.

Most values are concentrated on the main diagonal, which indicates correct classification. The remaining off-diagonal values represent visual confusion between digit classes.

8.5 Prediction examples

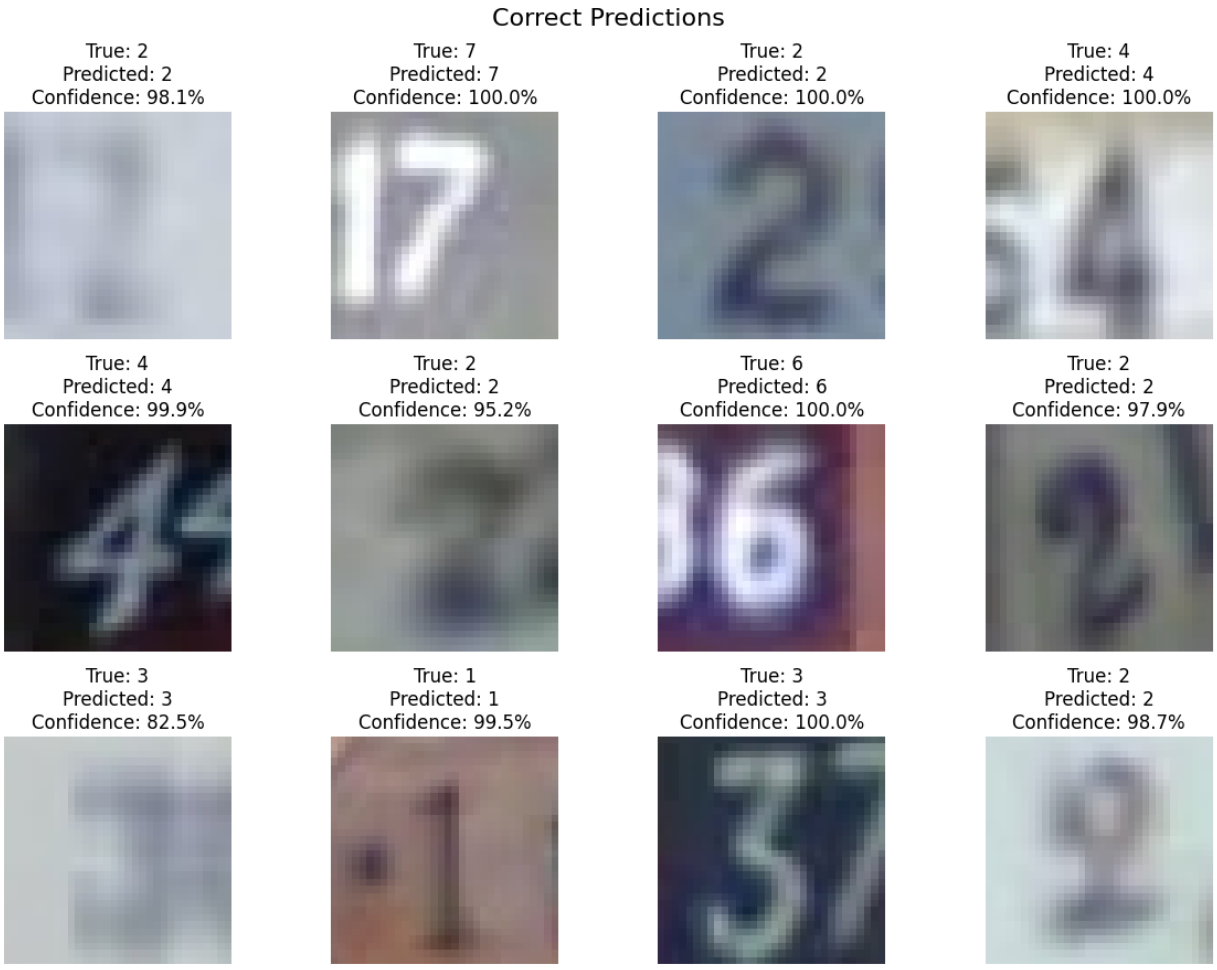


Figure 7. Examples of correct predictions.



Figure 8. Examples of incorrect predictions.

Incorrect cases commonly include blurred digits, neighboring numbers, weak contrast, partial crops, and shapes that resemble another digit.

9. Discussion

The results are suitable for the size and simplicity of the model. Two convolution layers were enough to learn useful visual features, and Dropout reduced the risk of memorizing the training set. The model did not show severe overfitting: training loss continued to decrease while validation loss remained close and reached its minimum near the end of training.

Class imbalance and image difficulty affect the results. Classes 1 and 2 contain many samples and achieved strong scores. Digit 8 has fewer training examples and frequently appears in difficult visual conditions, which may contribute to its weaker Recall. However, class size is not the only factor; visual similarity and crop quality also matter.

10. Limitations

- The system classifies one cropped digit image at a time.
- The model does not locate digits in a full street-view image.
- The extra SVHN training split was not used.
- Only one CNN architecture and one main learning rate were tested.
- The training distribution is not perfectly balanced.

11. Recommendations

- Use the SVHN extra split to increase training data.
- Apply small, realistic data augmentation such as limited translation or contrast changes.
- Compare a small number of filter sizes and learning rates.
- Use class weights only if further experiments confirm a serious class-imbalance problem.
- Extend the project later to detect and recognize multiple digits in complete images.

12. Conclusion

A simple CNN was successfully developed for SVHN digit classification. The complete workflow included dataset inspection, class analysis, data preparation, model construction, training, testing, and error analysis. The model achieved 87.92% test accuracy and a weighted F1-score of 0.8788. These results show that the selected approach is effective while remaining understandable and appropriate for a student machine learning project.